

Cheat Sheet: Building Unsupervised Learning Models

Unsupervised learning models

Model Name	Brief Description	Code Syntax
UMAP	UMAP (Uniform Manifold Approximation and Projection) is used for dimensionality reduction. Pros: High performance, preserves global structure. Cons: Sensitive to parameters. Applications: Data visualization, feature extraction. Key hyperparameters: <code>n_neighbors</code> : Controls the local neighborhood size (default = 15). <code>min_dist</code> : Controls the minimum distance between points in the embedded space (default = 0.1). <code>n_components</code> : The dimensionality of the embedding (default = 2).	<pre>1. from umap.umap_ import UMAP 2. umap = UMAP(n_neighbors=15, min_dist=0.1, n_components=2)</pre>
t-SNE	t-SNE (t-Distributed Stochastic Neighbor Embedding) is a nonlinear dimensionality reduction technique. Pros: Good for visualizing high-dimensional data. Cons: Computationally expensive, prone to overfitting. Applications: Data visualization, anomaly detection. Key hyperparameters: <code>n_components</code> : The number of dimensions for the output (default = 2). <code>perplexity</code> : Balances attention between local and global aspects of the data (default = 30). <code>learning_rate</code> : Controls the step size during optimization (default = 200).	<pre>1. from sklearn.manifold import TSNE 2. tsne = TSNE(n_components=2, perplexity=30, learning_rate=200)</pre>
PCA	PCA (principal component analysis) is used for linear dimensionality reduction. Pros: Easy to interpret, reduces noise. Cons: Linear, may lose information in nonlinear data. Applications: Feature extraction, compression. Key hyperparameters: <code>n_components</code> : Number of principal components to retain (default = 2). <code>whiten</code> : Whether to scale the components (default = False). <code>svd_solver</code> : The algorithm to compute the components (default = 'auto').	<pre>1. from sklearn.decomposition import PCA 2. pca = PCA(n_components=2)</pre>
DBSCAN	DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm. Pros: Identifies outliers, does not require the number of clusters. Cons: Difficult with varying density clusters. Applications: Anomaly detection, spatial data clustering. Key hyperparameters: <code>eps</code> : The maximum distance between two points to be considered neighbors (default = 0.5). <code>min_samples</code> : Minimum number of samples in a neighborhood to form a cluster (default = 5).	<pre>1. from sklearn.cluster import DBSCAN 2. dbscan = DBSCAN(eps=0.5, min_samples=5)</pre>
HDBSCAN	HDBSCAN (Hierarchical DBSCAN) improves on DBSCAN by handling varying density clusters. Pros: Better handling of varying densities. Cons: Can be slower	<pre>1. import hdbscan 2. clusterer = hdbscan.HDBSCAN(min_cluster_size=</pre>

	varying densities.Cons: Can be slower than DBSCAN.Applications: Large datasets, complex clustering problems.Key hyperparameters:min_cluster_size: The minimum size of clusters (default = 5).min_samples: Minimum number of samples to form a cluster (default = 10).	
K-Means clustering	K-Means is a centroid-based clustering algorithm that groups data into k clusters.Pros: Efficient, simple to implement.Cons: Sensitive to initial cluster centroids.Applications: Customer segmentation, pattern recognition.Key hyperparameters:n_clusters: Number of clusters (default = 8).init: Method for initializing the centroids ('k-means++' or 'random', default = 'k-means++').n_init: Number of times the algorithm will run with different centroid seeds (default = 10).	<pre>1. from sklearn.cluster import KMean 2. kmeans = KMeans(n_clusters=3)</pre>

Associated functions used

Method	Brief Description	Code Syntax
make_blobs	Generates isotropic Gaussian blobs for clustering.	<pre>1. from sklearn.datasets import make_blobs 2. X, y = make_blobs(n_samples=100, centers=2, random_state=42)</pre>
multivariate_normal	Generates samples from a multivariate normal distribution.	<pre>1. from numpy.random import multivariate_normal 2. samples = multivariate_normal(mean=[0, 0], cov=[[1, 0], [0, 1]], size=100)</pre>
plotly.express.scatter_3d	Creates a 3D scatter plot using Plotly Express.	<pre>1. import plotly.express as px 2. fig = px.scatter_3d(df, x='x', y='y', z='z') 3. fig.show()</pre>
geopandas.GeoDataFrame	Creates a GeoDataFrame from a Pandas DataFrame.	<pre>1. import geopandas as gpd 2. gdf = gpd.GeoDataFrame(df, geometry='geometry')</pre>
geopandas.to_crs	Transforms the coordinate reference system of a GeoDataFrame.	<pre>1. gdf = gdf.to_crs(epsg=3857)</pre>
contextily.add_basemap	Adds a basemap to a GeoDataFrame plot for context.	<pre>1. import contextily as ctx 2. ax = gdf.plot(figsize=(10, 10)) 3. ctx.add_basemap(ax)</pre>
pca.explained_variance_ratio_	Returns the proportion of variance explained by each principal component.	<pre>1. from sklearn.decomposition import PCA 2. pca = PCA(n_components=2) 3. pca.fit(X) 4. variance_ratio = pca.explained_variance_ratio_</pre>

Author

[Jeff Grossman](#) 

[Abhishek Gagneja](#) 

Go to next item

✓ Completed

👍 Like

👎 Dislike

🚩 Report an issue

Go to next item →